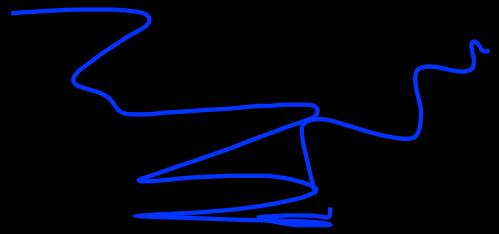


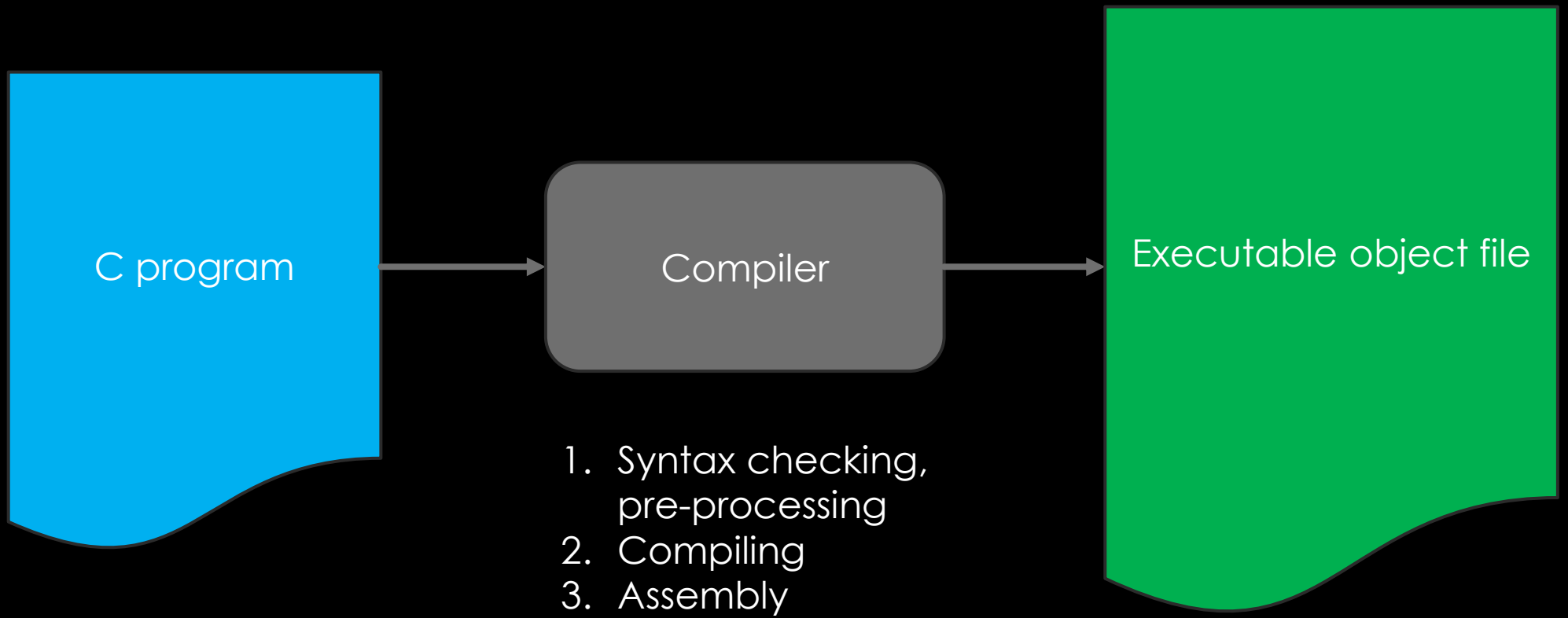
LECTURE 2

Language categorization; Printing, Storing & Inputting Numbers; Math Expressions & Library

RECAP

- Digital Computers
  - Programmable
  - Run programs interleaved
  - Compose programs with programs

RECAP



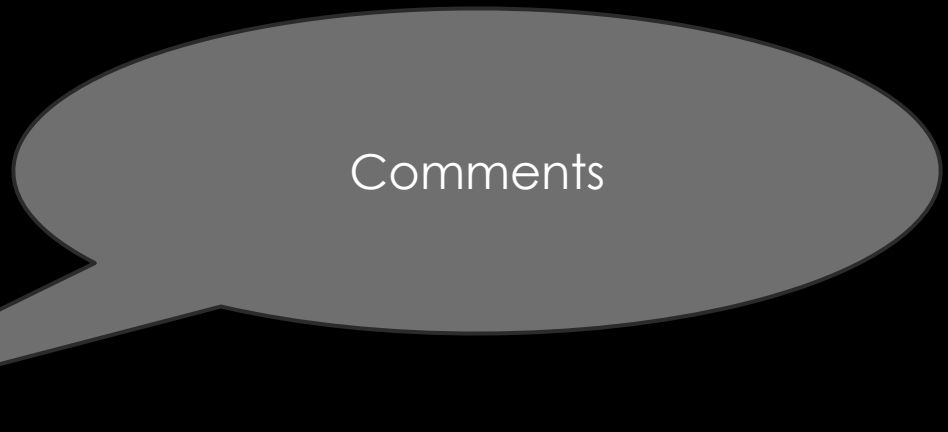
1. Syntax checking, pre-processing
2. Compiling
3. Assembly
4. Linking

HELLO WORLD!

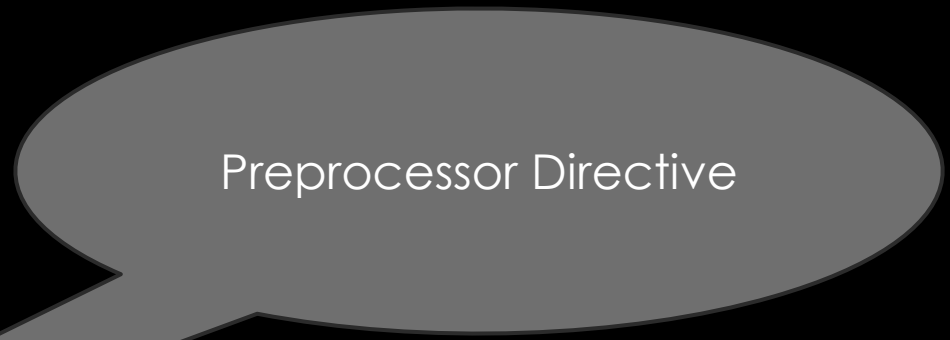
```c
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

Syntax

Rules of C programming language
Required for compiler to understand the program in c and convert to machine language



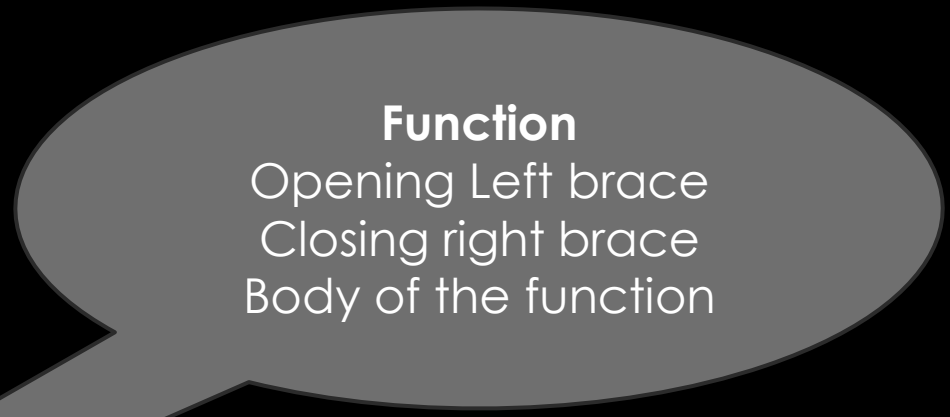

```c
/* A C program to print “Hello World!”
    on the computer screen. */
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```




```c
/* A C program to print “Hello Wo    .
    on the computer screen    /
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```

Function
Program begins execution
at the **main()** function
Int : return value
Void : no arguments

```c
/* A C program to print “Hello Wor
    on the computer screen
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```




```c
/* A C program to print “Hello World!”
    on the computer screen. */
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```

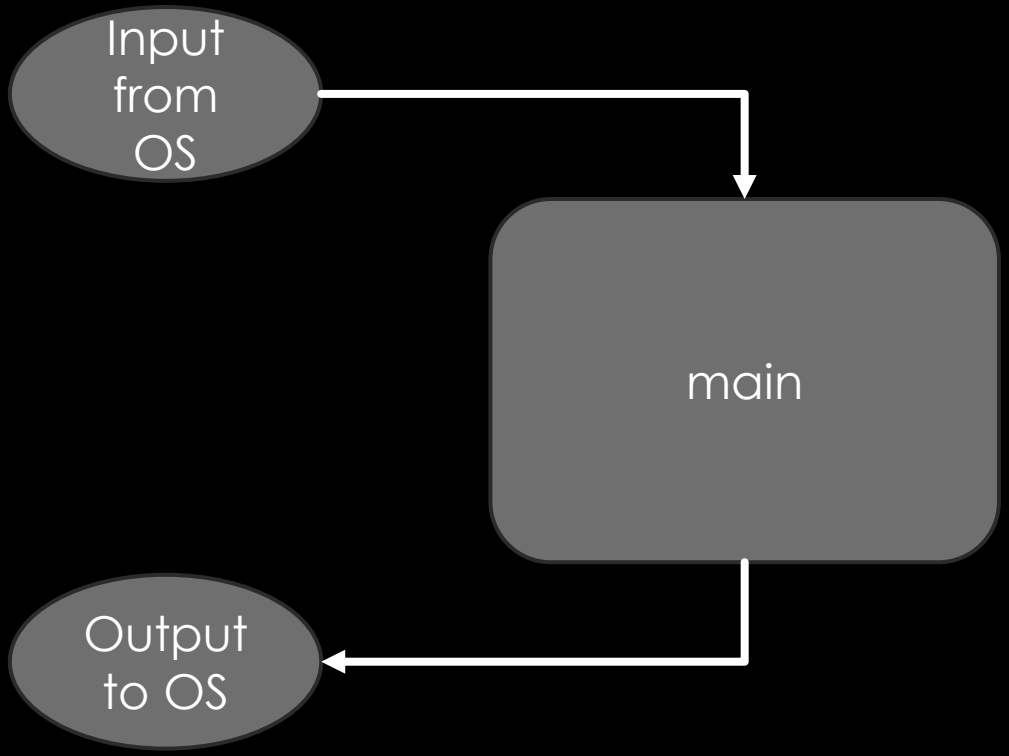
Body or Block

```c
/* A C program to print “Hello World!”
    on the computer screen. */
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```

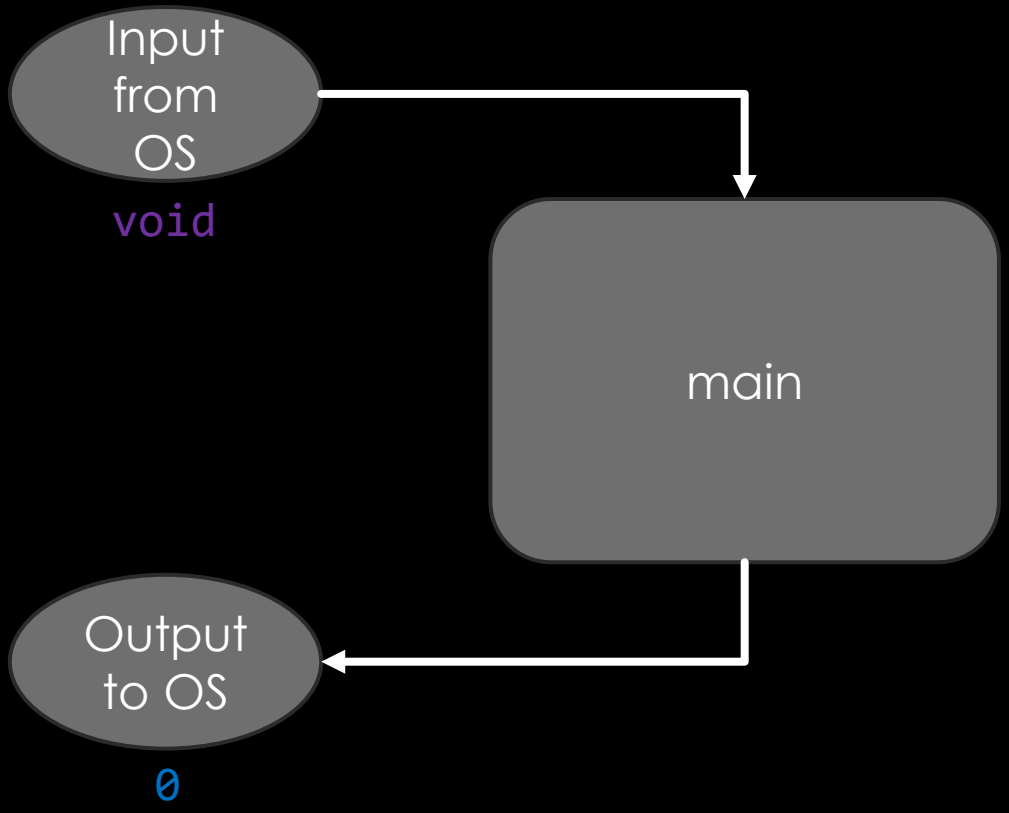
- Statement
- Functional call with an argument
- Message or character string with escape character

Escape sequence	Description
\n	Moves the cursor to the beginning of the next line.
\t	Moves the cursor to the next horizontal tab stop.
\a	Produces a sound or visible alert without changing the current cursor position.
\\	Because the backslash has special meaning in a string, \\ is required to insert a backslash character in a string.
\"	Because strings are enclosed in double quotes, \" is required to insert a double-quote character in a string.

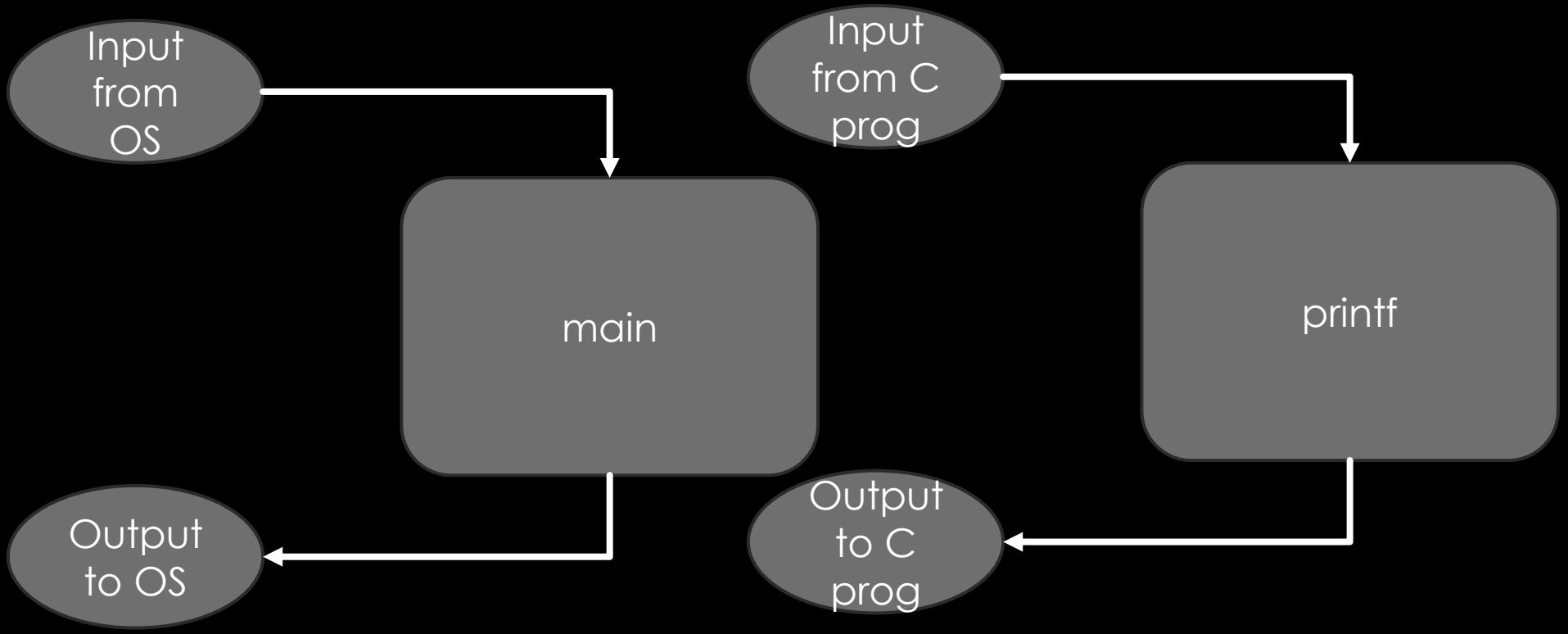
FUNCTIONAL VIEW



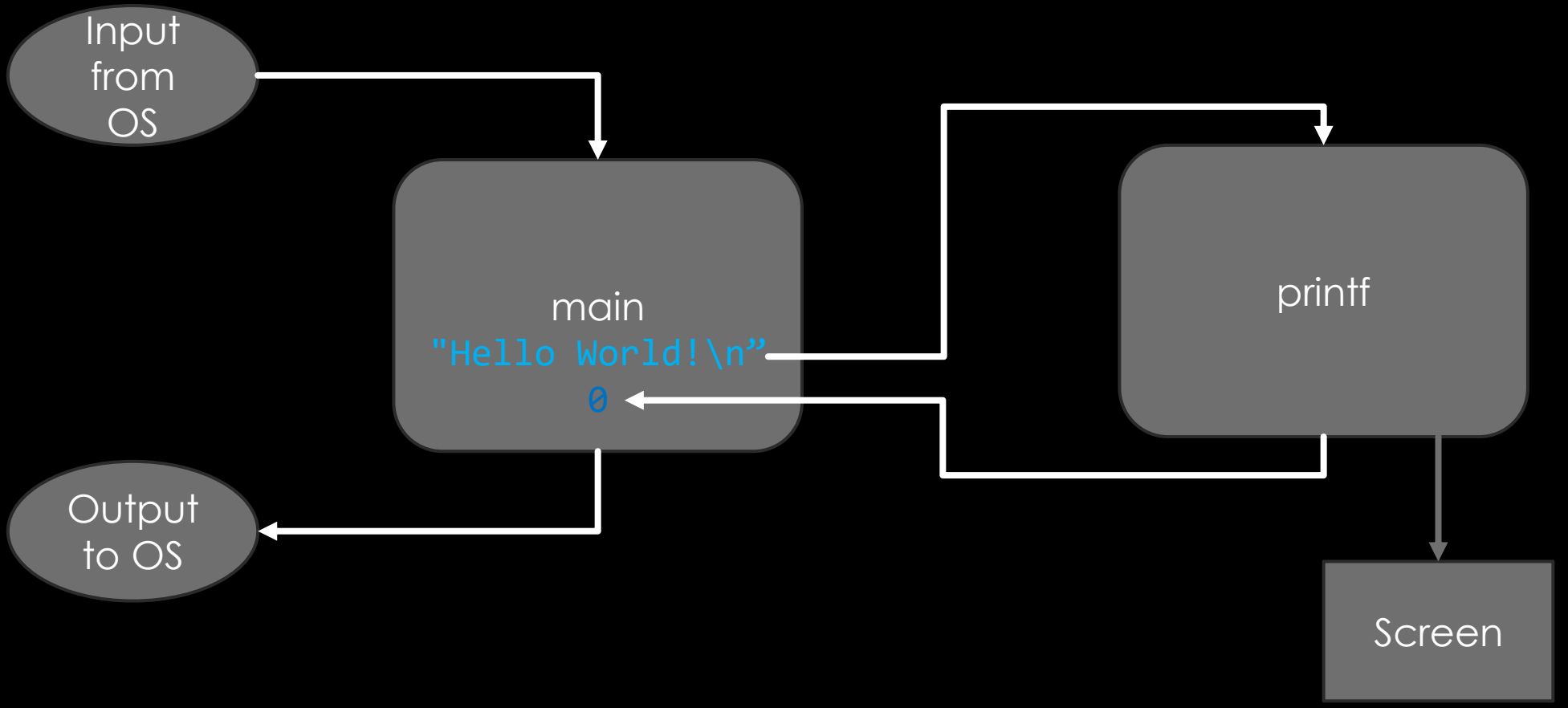
FUNCTIONAL VIEW



FUNCTIONAL VIEW



FUNCTIONAL VIEW



TIP: INDENTATION

```c
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0;
}
```

```c
// fig02_02.c
// Printing on one line with two printf statements.
#include <stdio.h>

// function main begins program execution
int main(void) {
   printf("Welcome ");
   printf("to C!\n");
} // end function main
```


```c
// fig02_02.c
// Printing on one line with two printf statements.
#include <stdio.h>

// function main begins program execution
int main(void) {
   printf("Welcome ");
   printf("to C!\n");
} // end function main
```

ADDING TWO INTEGERS

Variables and Variable Definitions

Type : Int

- Memory is allocated to hold integer type
- -Initialization not necessary

-All variables must be defined

```
1  // fig02_04.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int integer1 = 0; // will hold first number user enters
8     int integer2 = 0; // will hold second number user enters
9
10    printf("Enter first integer: "); // prompt
11    scanf("%d", &integer1); // read an integer
12
13    printf("Enter second integer: "); // prompt
14    scanf("%d", &integer2); // read an integer
15
16    int sum = 0; // variable in which sum will be stored
17    sum = integer1 + integer2; // assign total to sum
18
19    printf("Sum is %d\n", sum); // print sum
20 } // end function main
```

ADDING TWO INTEGERS

Variable Name
- *letter, digit, underscore*
- *C is case sensitive*
- *Must not begin with digit*

total_commissions
totalCommissions

```
1  // fig02_04.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int integer1 = 0; // will hold first number user enters
8     int integer2 = 0; // will hold second number user enters
9
10    printf("Enter first integer: "); // prompt
11    scanf("%d", &integer1); // read an integer
12
13    printf("Enter second integer: "); // prompt
14    scanf("%d", &integer2); // read an integer
15
16    int sum = 0; // variable in which sum will be stored
17    sum = integer1 + integer2; // assign total to sum
18
19    printf("Sum is %d\n", sum); // print sum
20 } // end function main
```

ADDING TWO INTEGERS

```
1  // fig02_04.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execu
6  int main(void) {
7     int integer1 = 0; // will h      st number user enters
8     int integer2 = 0; // w      d second number user enters
9
10       printf("Enter first integer: "); // prompt
11       scanf("%d", &integer1); // read an integer
12
13       printf("Enter second integer: "); // prompt
14       scanf("%d", &integer2); // read an integer
15
16       int sum = 0; // variable in which sum will be stored
17       sum = integer1 + integer2; // assign total to sum
18
19       printf("Sum is %d\n", sum); // print sum
20 } // end function main
```

- Scanf function
- Obtain value from user
- Reads value from std input
- Consider 2 arguments
- %d : format - type of data
- &integer1 : location or address in memory

Segmentation error !

ADDING TWO INTEGERS

```
1  // fig02_04.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int integer1 = 0; // will hold first n          enters
8     int integer2 = 0; // will hold seco             user enters
9
10       printf("Enter first inte          ); // prompt
11       scanf("%d", &integer1          read an integer
12
13       printf("Enter s          d integer: "); // prompt
14       scanf("%d", &          teger2); // read an integer
15
16       int sum = 0; // variable in which sum will be stored
17       sum = integer1 + integer2; // assign total to sum
18
19       printf("Sum is %d\n", sum); // print sum
20 } // end function main
```

Assignment statement
= assignment operator
Binary operator =, +

ADDING TWO INTEGERS

```
1  // fig02_04.c
2  // Addition program.
3  #include <stdio.h>
4
5  // function main begins program execution
6  int main(void) {
7     int integer1 = 0; // will hold first number user enters
8     int integer2 = 0; // will hold second number user ent
9
10    printf("Enter first integer: "); // prompt
11    scanf("%d", &integer1); // read an intege
12
13    printf("Enter second integer: "); //    mpt
14    scanf("%d", &integer2); // read an    teger
15
16    int sum = 0; // variable in whi   sum will be stored
17    sum = integer1 + integer2; //  assign total to sum
18
19    printf("Sum is %d\n", sum); // print sum
20 } // end function main
```

Printing with format control string
%d is placeholder for integer

printf("Sum is %d\n", integer1 + integer2);

PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
    printf("234+456 = %d\n",234+456);
    // %d or %i is format specifier (decimal integer)
    return 0;
}
```

SPOT THE ERROR!

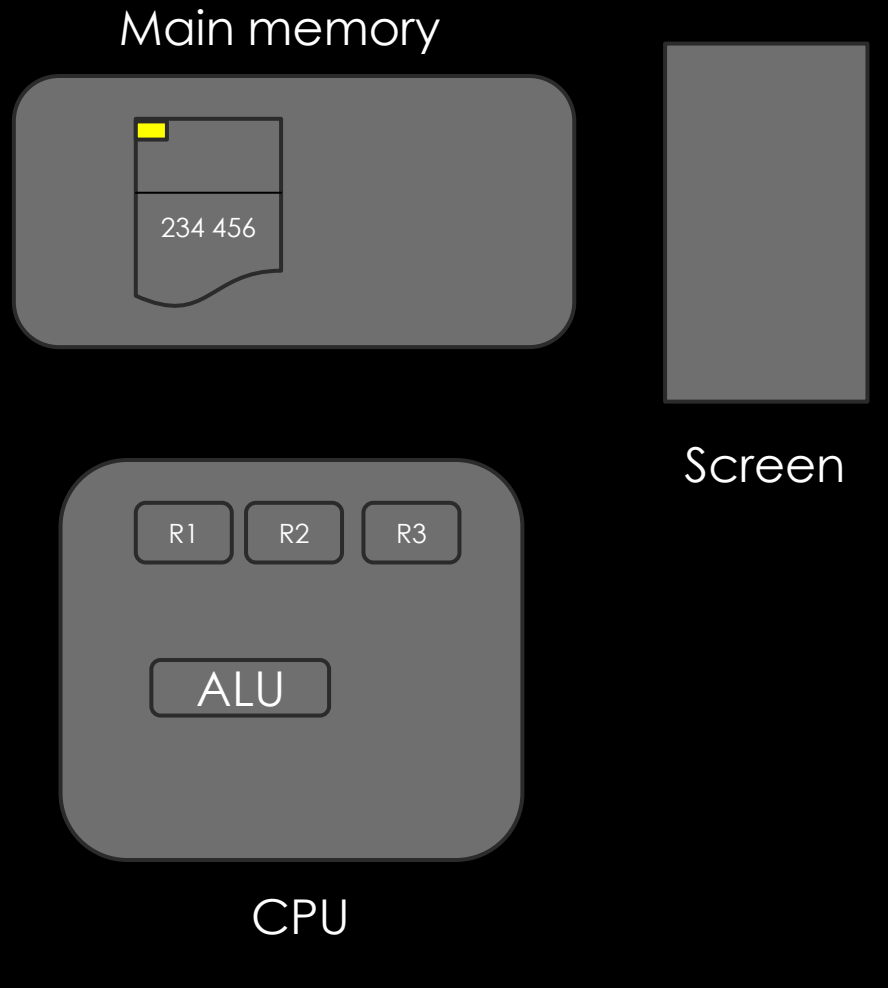
```
/* A C program to print “Hello World!”
    on the computer screen.
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```

SPOT THE ERROR!

```
/* A C program to print “Hello World!”
    on the computer screen.
#include <stdio.h>

int main(void) {
    printf("Hello World!\n");
    return 0; // `0’ to indicate program ran successfully
}
```

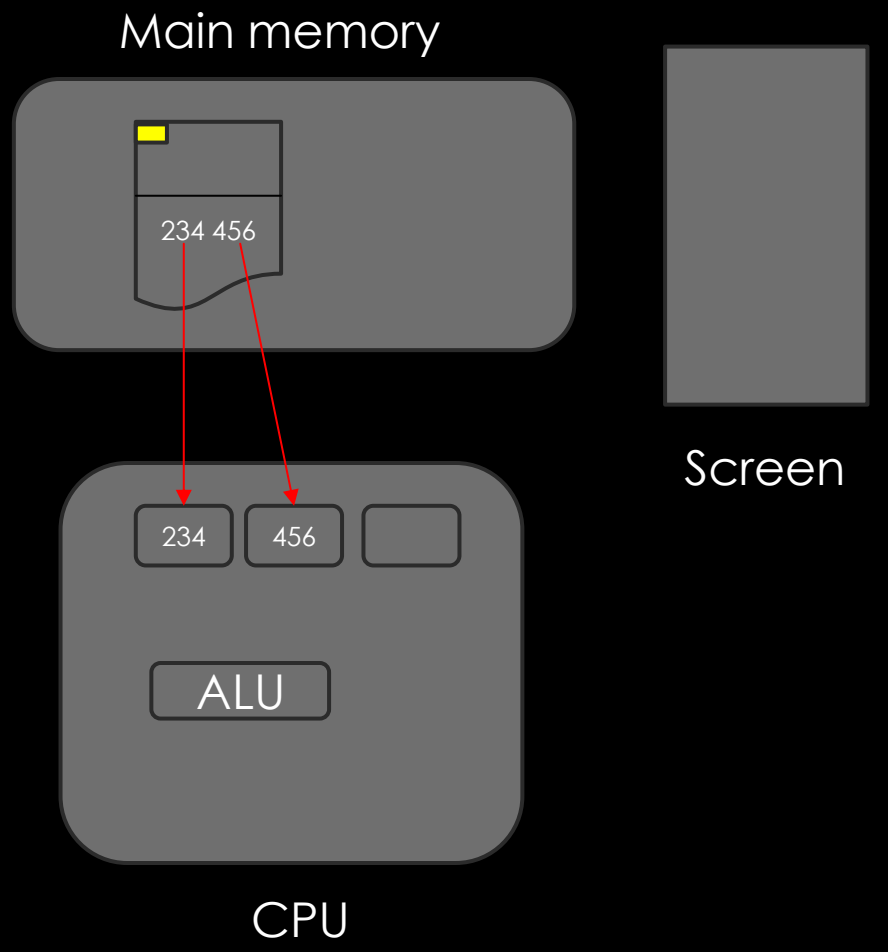
PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
     printf("234+456 = %d\n",234+456);
    return 0;
}
```



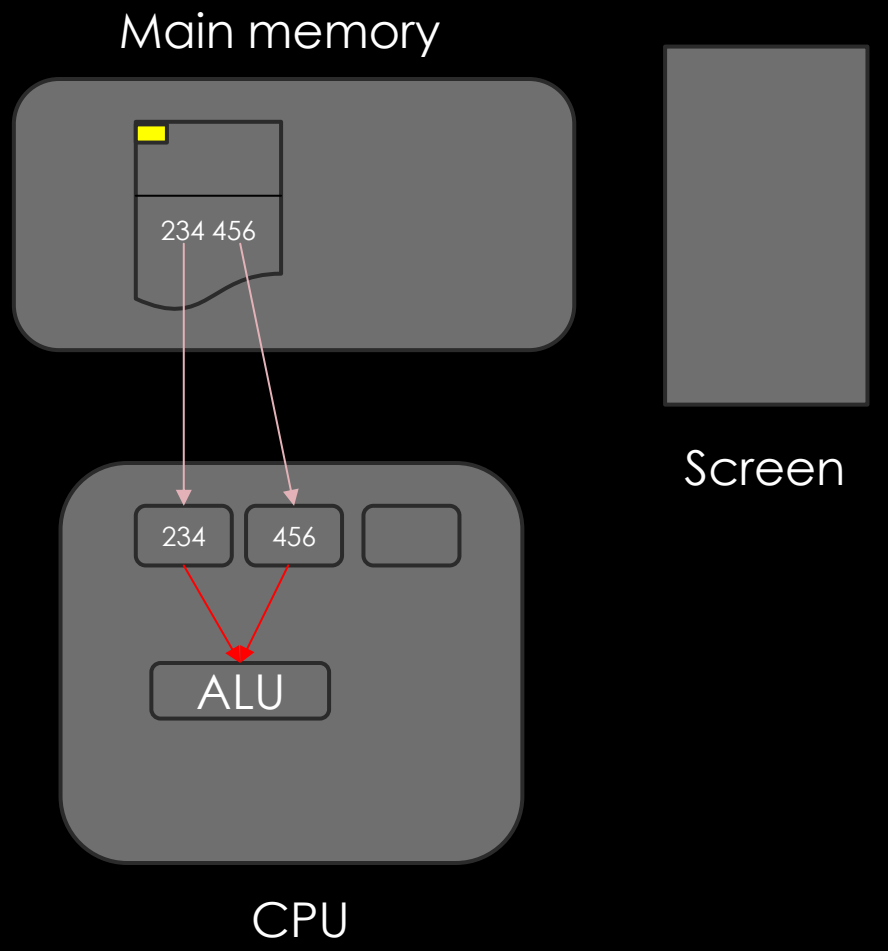
PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
     printf("234+456 = %d\n",234+456);
    return 0;
}
```



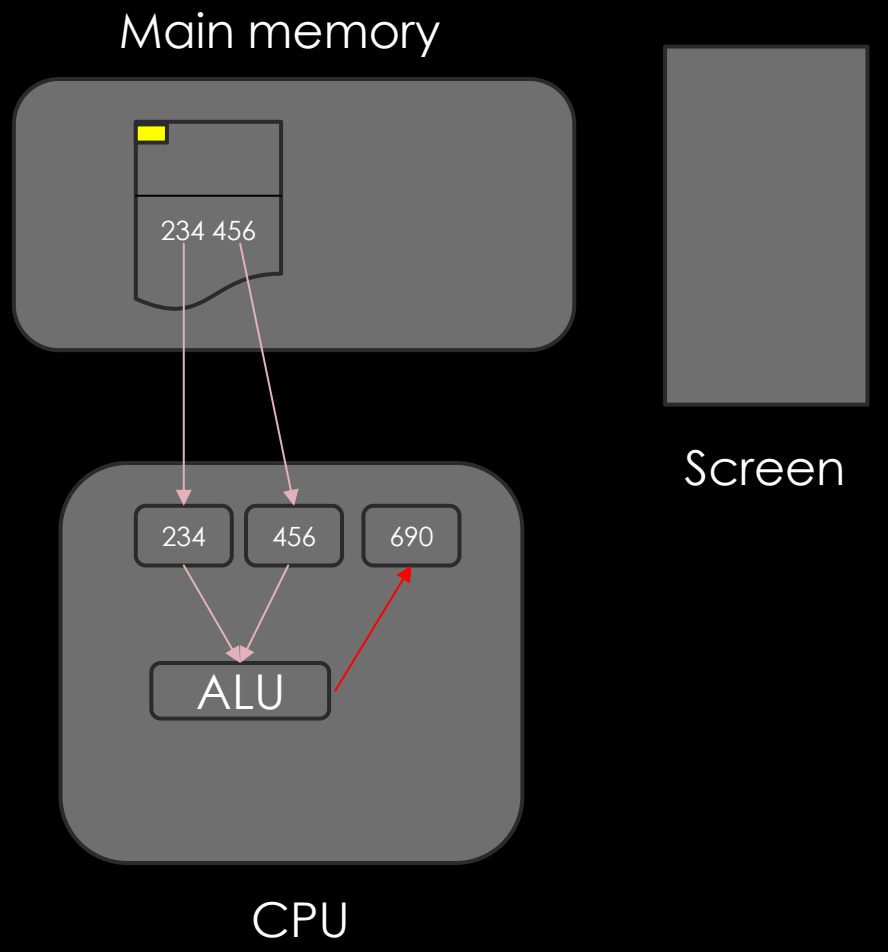
PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
    printf("234+456 = %d\n",234+456);
    // %d is format specifier
    return 0;
}
```



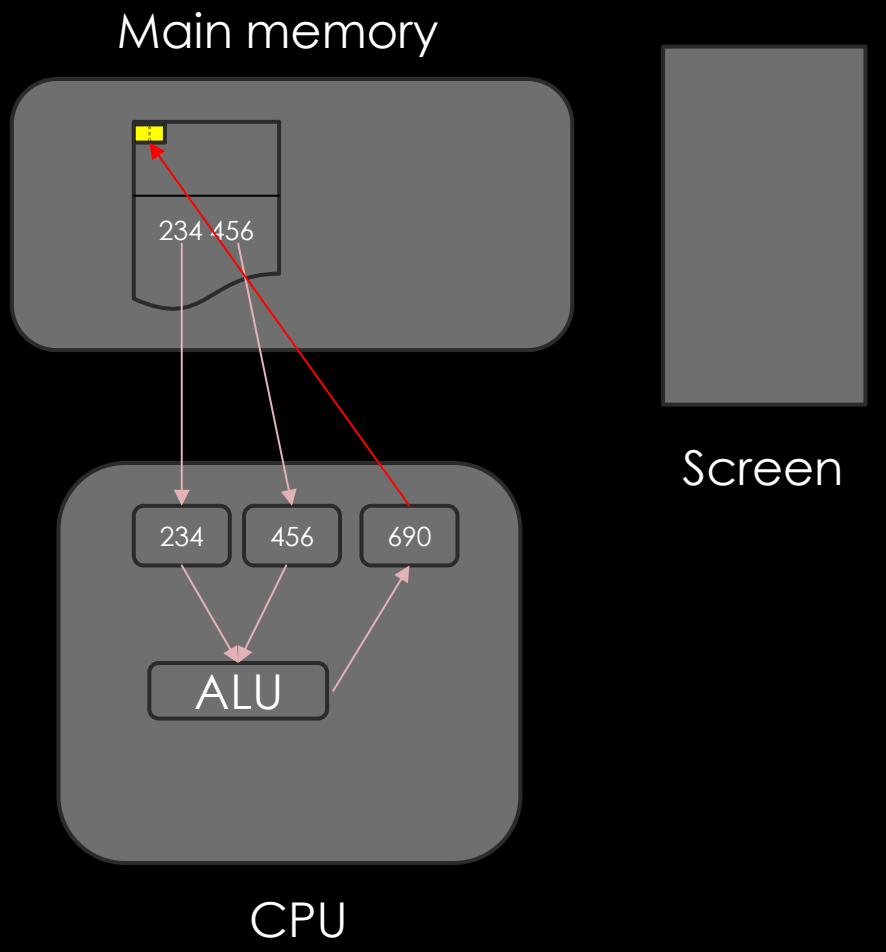
PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
     printf("234+456 = %d\n",234+456);
    return 0;
}
```



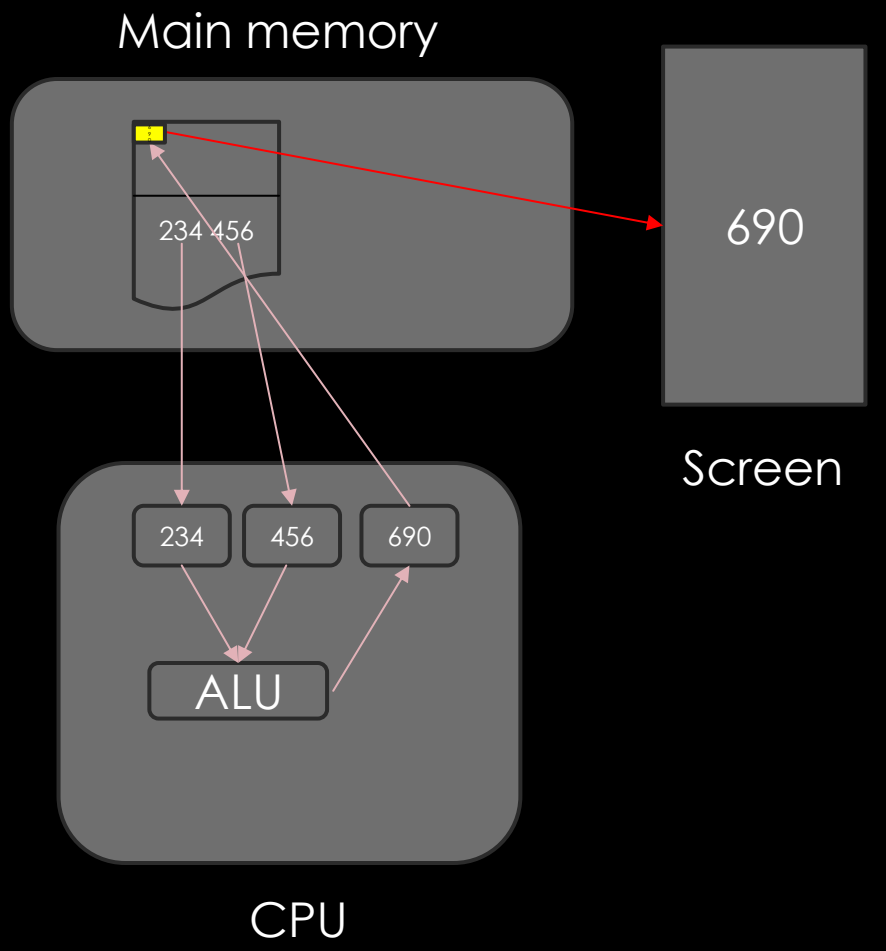
PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
     printf("234+456 = %d\n",234+456);
    return 0;
}
```



PRINTING AN EXPRESSION'S VALUE

```c
#include <stdio.h>

int main(void) {
     printf("234+456 = %d\n",234+456);
    return 0;
}
```



ARITHMETIC IN C

C operation	Arithmetic operator	Algebraic expression	C expression
Addition	+	$f + 7$	`f + 7`
Subtraction	-	$p - c$	`p - c`
Multiplication	*	bm	`b * m`
Division	/	x / y or $\frac{x}{y}$	`x / y`
Remainder	%	$r \bmod s$	`r % s`

RULES OF OPERATOR PRECEDENCE IN C

1. Expressions grouped in parentheses evaluate first. Parentheses are said to be at the "highest level of precedence." In nested parentheses, such as

   ```
   ((a + b) + c)
   ```

   the operators in the innermost pair of parentheses are applied first.
2. *, / and % are applied next. If an expression contains several *, / and % operators, evaluation proceeds left-to-right. These three operators are said to be on the same level of precedence.
3. + and - are evaluated next. If an expression contains + and - operators, evaluation proceeds left-to-right. These two operators have the same level of precedence, which is lower than that of *, / and %.
4. The assignment operator (=) is evaluated last.

RULES OF OPERATOR PRECEDENCE IN C

Average computation : m = a+b+c+d+e/5

RULES OF OPERATOR PRECEDENCE IN C

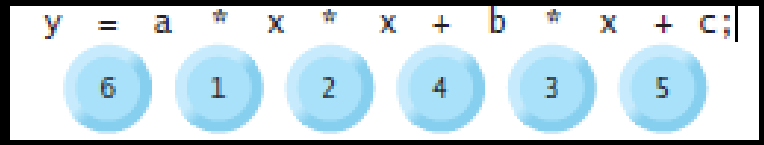
Average computation :

m = a+b+c+d+e/5;

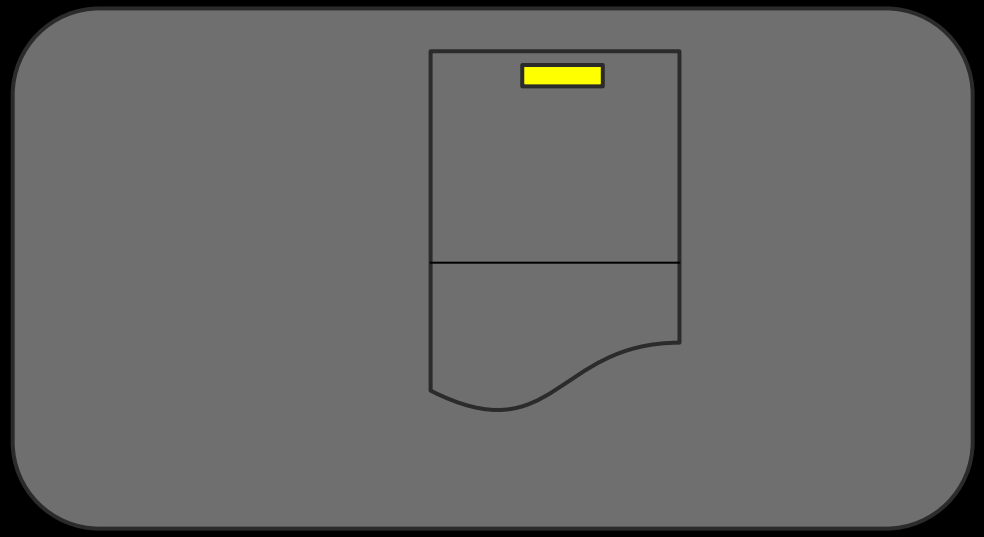
m = (a+b+c+d+e)/5;

Equation of straight line y = m*x+b;

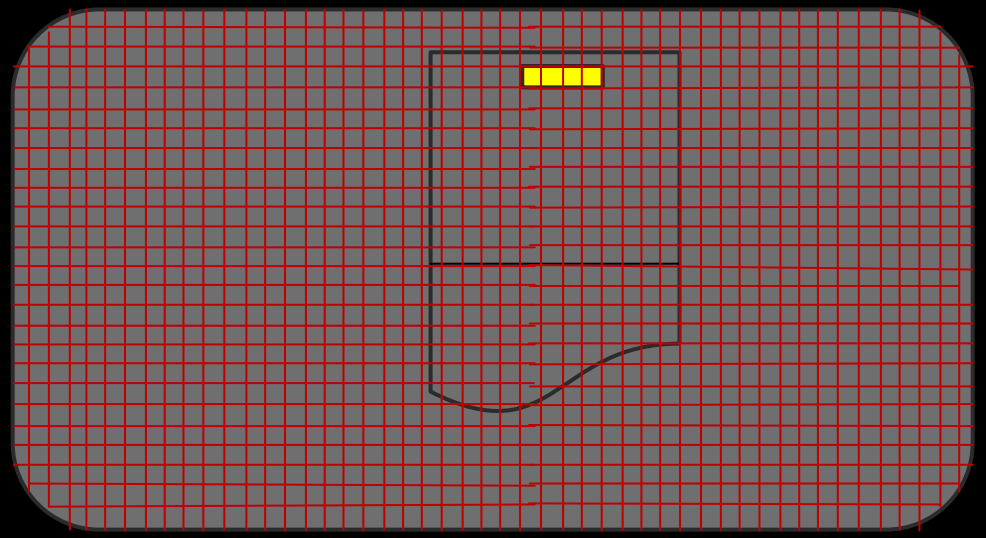
second degree polynomial y = a * x * x + b * x + c;



INTERNAL REPRESENTATION (BINARY)

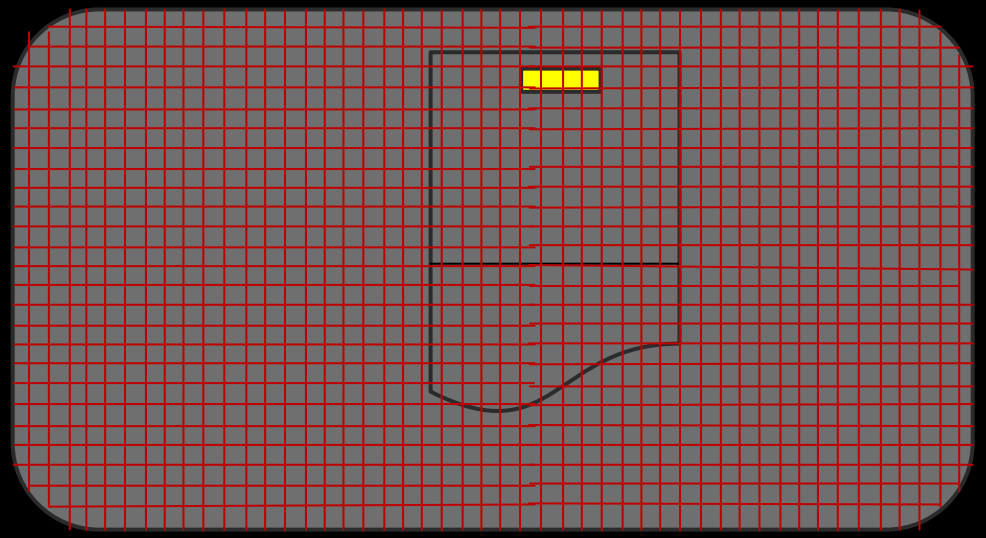


INTERNAL REPRESENTATION (BINARY)



- Memory is array of bits
- 8 bits is one byte

INTERNAL REPRESENTATION (BINARY)



- Memory is array of bits
- 8 bits is one byte
- Same sequence of bits interpreted
  - instructions
  - data
    - decimal integers
    - decimal rationals
    - characters

INTERPRETING AS NUMBERS

10001001

- Number interpretation is well known
  - Binary system
  - $= 1 * 2^7 + 1 * 2^3 + 1 * 2^0 = 137$

INTERPRETING AS INTEGERS

10001001

- Integer interpretation

INTERPRETING AS INTEGERS

10001001

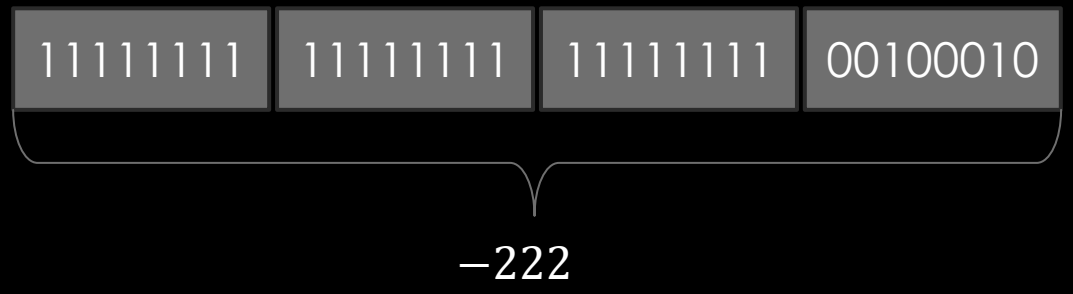
- Integer interpretation
  - Binary system, reserve 1 bit for sign
    - $a_7a_6a_5a_4a_3a_2a_1a_0 = (-1)^{a_7} \sum_{i=0}^{6} a_i 2^i$
    - $= -(1 * 2^3 + 1 * 2^0)$= -9

INTERPRETING AS INTEGERS

10001001

- Integer interpretation
  - Binary system, reserve 1 bit for sign
    - $a_7a_6a_5a_4a_3a_2a_1a_0 = (-1)^{a_7} \sum_{i=0}^{6} a_i 2^i$
    - $= -(1 * 2^3 + 1 * 2^0)$= -9

`int` DATA TYPE

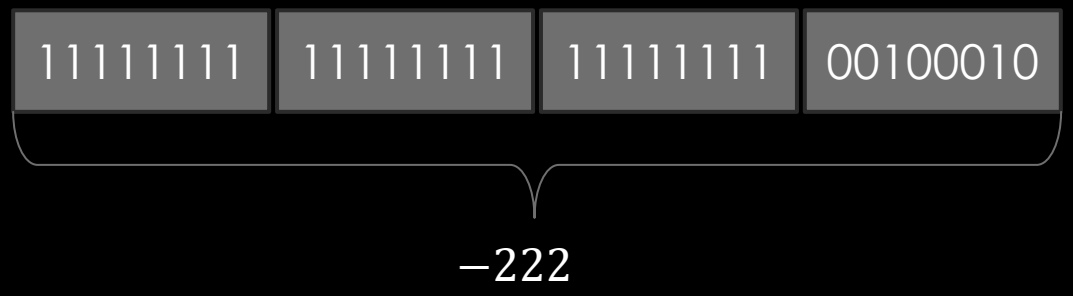


- Four bytes
- -2^{31} to $2^{31} - 1$

Compute Two's Complement (to represent −N):
1.Start with the binary representation of **positive N**.
2.Invert all bits (flip 0s and 1s).
3.Add 1 to the result.

- To represent -5
- Binary of 5 => 00000101
- Invert bits → 11111010
- Add 1 → 11111011

`int` DATA TYPE



- Four bytes
- -2^{31} to $2^{31} - 1$

- The same **binary addition circuits** work for **both positive and negative numbers**.
- You don't need separate logic for signed and unsigned operations.

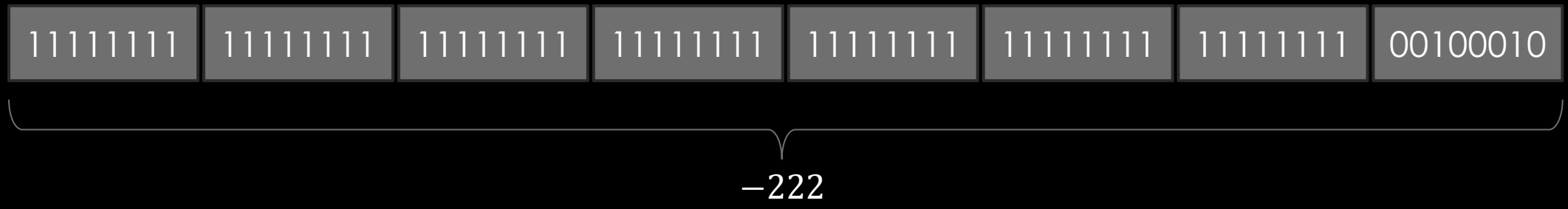
Example:

`5 + (-3)`

is just binary addition of `00000101 + 11111101` = `00000010` → 2

`long int` QUALIFIED DATA TYPE

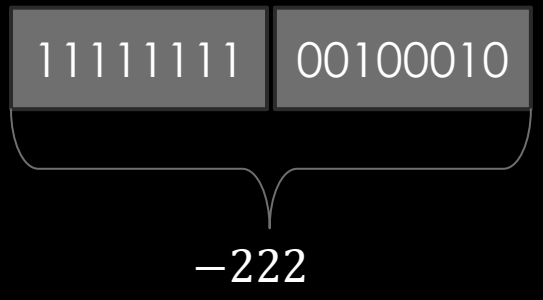
```
long int diff; // define a 8-byte integer variable
```



- Eight bytes
- -2^{63} to $2^{63} - 1$

`short int` QUALIFIED DATA TYPE

```
short int diff; // define a 2-byte integer variable
```



- 2 bytes
- -2^{15} to $2^{15} - 1$

VARIOUS int DATATYPE QUALIFIERS

Data Type	Bytes	Range	Specifier
short int	2	-2^{15} to $2^{15}-1$	%hd
int	4	-2^{31} to $2^{31}-1$	%d
long int	8	-2^{63} to $2^{63}-1$	%ld
short unsigned int	2	0 to $2^{16}-1$	%hu
unsigned int	4	0 to $2^{32}-1$	%u
long unsigned int	8	0 to $2^{64}-1$	%lu

GUESS OUTPUT

```c
#include <stdio.h>

int main(void) {
    int diff;
    diff = 234-456;
    printf("234-456=%u\n",diff); // %u specifies interpret as 4-byte number
    return 0;
}
```

GUESS OUTPUT

```c
#include <stdio.h>

int main(void) {
    int diff;
    diff = 2147483648;
    printf("%d",diff);
    return 0;
}
```

MOTION WITH CONSTANT ACCELERATION

```c
#include <stdio.h>

int main(void) {
    int initialVelocity=10, acceleration=5, duration=3, finalVelocity;
    // You can define multiple variables at a time
    // You can initialize values while defining
    finalVelocity = initialVelocity + acceleration*duration;
    printf("Final velocity of the particle = %d\n", finalVelocity);
    return 0;
}
```

MOTION WITH CONSTANT ACCELERATION

```c
#include <stdio.h>

int main(void) {
    int initialVelocity=10, acceleration=5, duration=3, finalVelocity;
    finalVelocity = initialVelocity + acceleration*duration;
    printf("Final velocity of the particle = %d\n", finalVelocity);
    return 0;
}
```

Representation of Rationals

- Trade off between range and precision
- Astronomical numbers don't require great precision
- Microscopic numbers don't require large range
- Few significant digits only matter for arithmetic!

REPRESENTATION OF RATIONALS

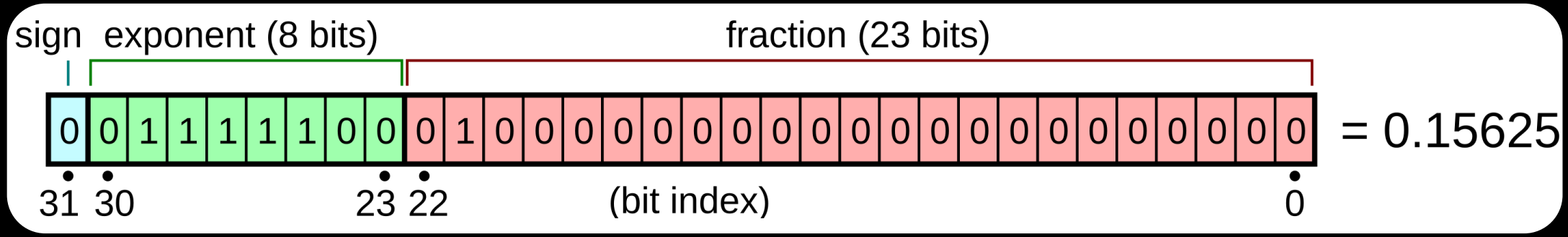
$6.02214076e^{23}$

$1.380649e^{-23}$

$6.6743e^{-11}$

$3e^{8}$

`float` DATA TYPE



IEEE 754 single-precision format

`float` DATA TYPE

Adjective	Value
Largest number below 1	$0.99999994039535522 5$
Smallest number above 1	$1.0000001192092895 5$
Largest number	$3.4028234664 \times 10^{38}$
Smallest positive number	$1.4012984643 \times 10^{-45}$

VARIOUS RATIONAL NUMBER DATATYPES

Data Type	Bytes	Smallest Positive	Largest	Specifier
`float`	4	1.4×10^{-45}	3.4×10^{38}	`%f`
`double`	8	4.9×10^{-324}	1.8×10^{308}	`%lf`

MOTION WITH CONSTANT ACCELERATION

```c
#include <stdio.h>

int main(void) {
    float initialVelocity=10, acceleration=5, duration=3, displacement;
    displacement = initialVelocity*duration + 0.5*acceleration*duration*duration;
    printf("Displacement of the particle = %f\n", displacement);
    return 0;
}
```

MOTION WITH CONSTANT ACCELERATION

Useful if obtained from user or OS

```c
#include <stdio.h>

int main(void) {
    float initialVelocity=10, acceleration=5, duration=3, displacement;
    displacement = initialVelocity*duration + 0.5*acceleration*duration*duration;
    printf("Displacement of the particle = %f\n", displacement);
    return 0;
}
```

MOTION WITH CONSTANT ACCELERATION

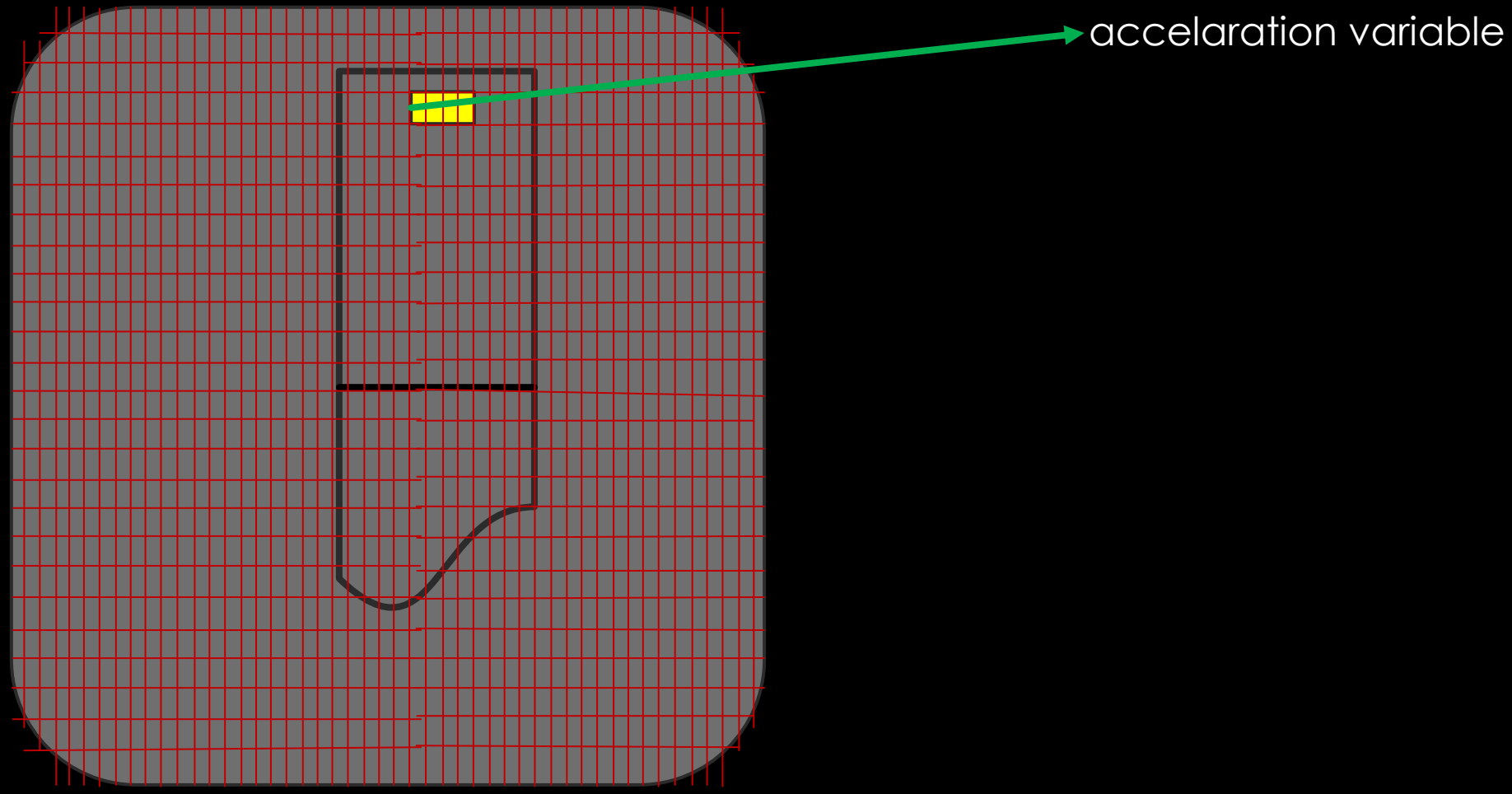
```c
#include <stdio.h>

int main(void) {
    float initialVelocity=10, acceleration=5, duration=3, displacement;
    displacement = initialVelocity*duration + 0.5*acceleration*duration*duration;
    printf("Displacement of the particle = %f\n", displacement);
    return 0;
}
```

Receive from OS
(covered later)

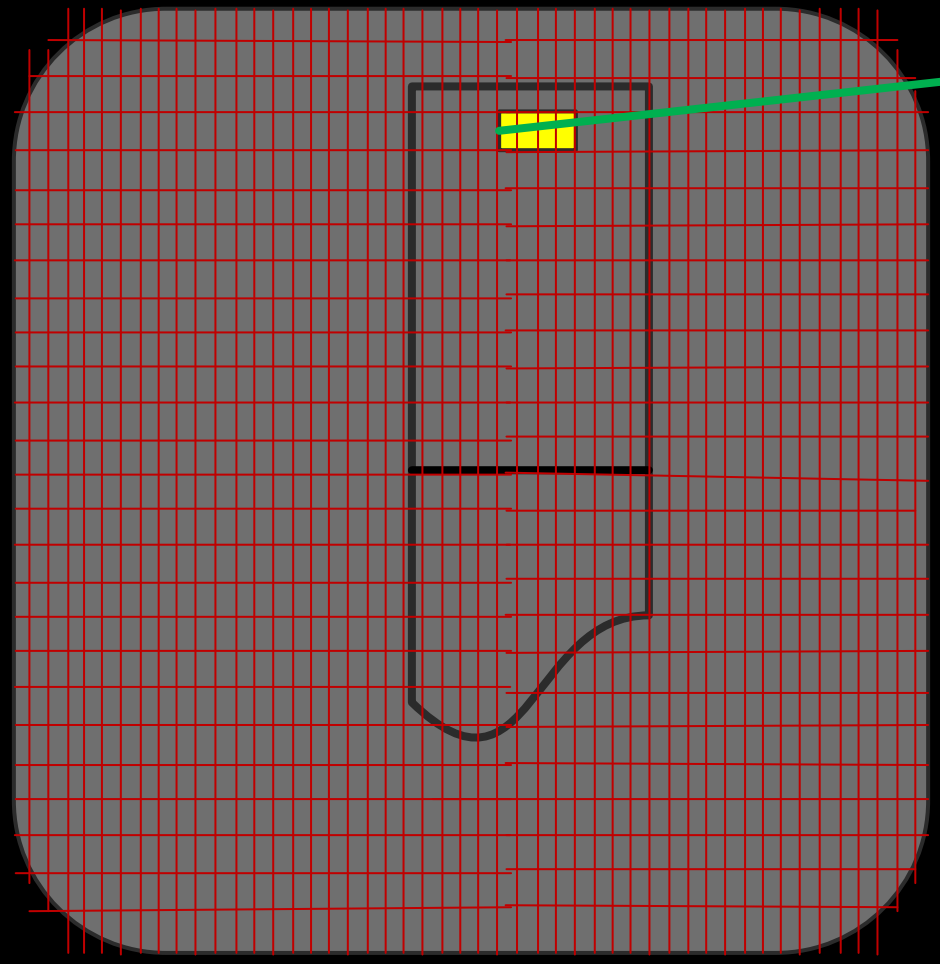
TAKING INPUT FROM USER/KEYBOARD

```c
#include <stdio.h>

int main(void) {
    float initialVelocity, acceleration, duration, displacement;
    printf("Enter initialVelocity: ");            //Prompt user to enter
    scanf("%f",&initialVelocity);                 //store keyboard entered rational
    printf("\nEnter acceleration: ");
    scanf("%f",&acceleration);
    printf("\nEnter duration of motion: ");
    scanf("%f",&duration);
    displacement = initialVelocity*duration + 0.5*acceleration*duration*duration;
    printf("Displacement of the particle = %f\n", displacement);
    return 0;
}
```

& OPERATOR



& OPERATOR

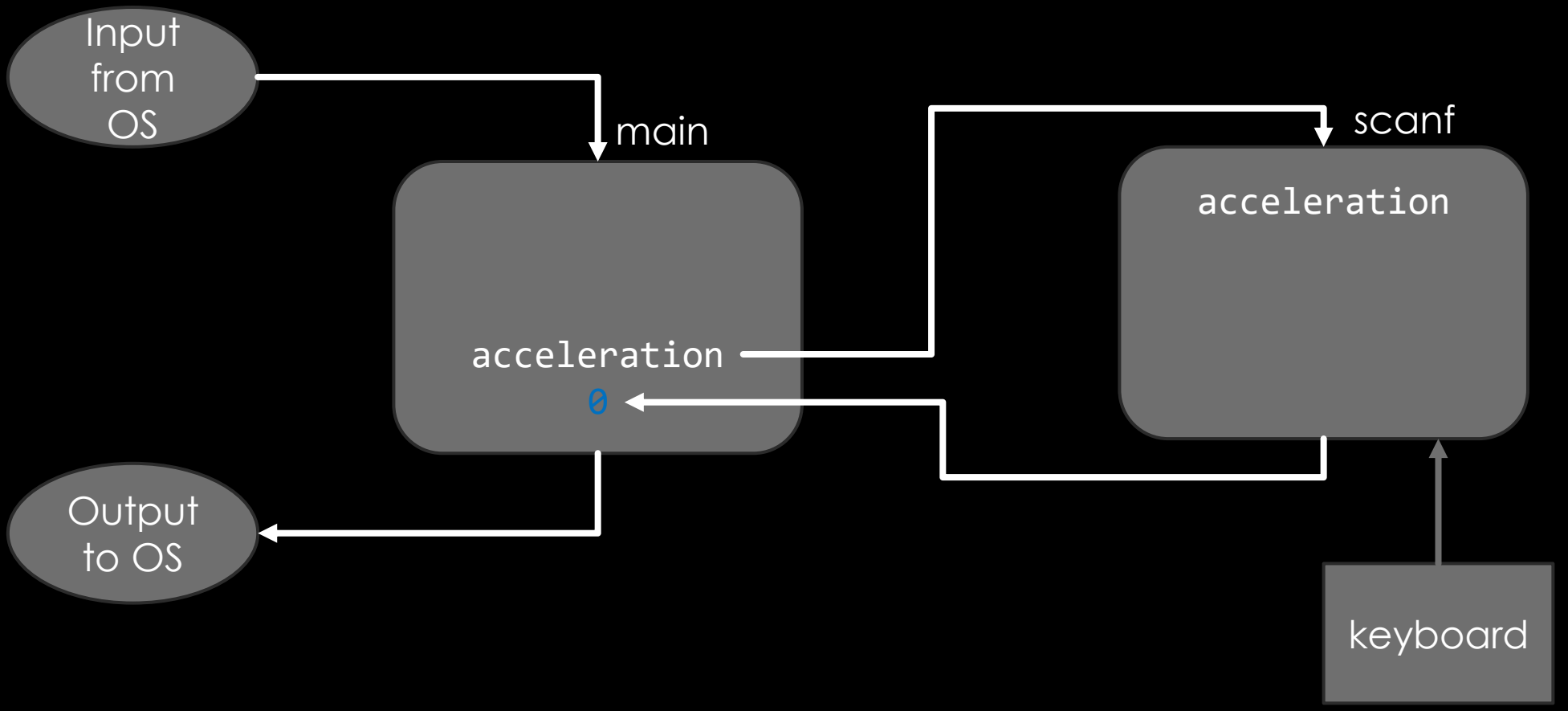


&accelearation returns the (relative) address of first byte of this variable in memory

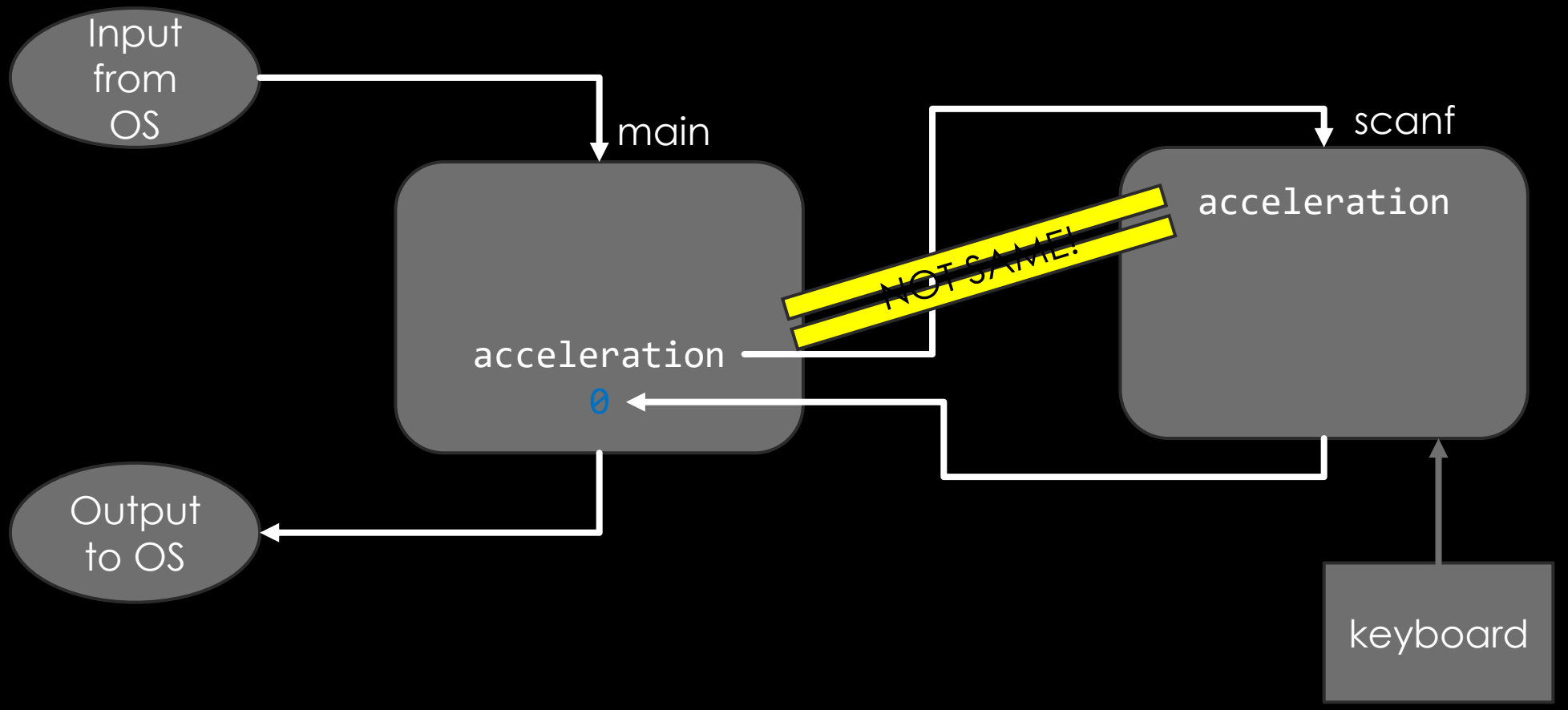
GUESS OUTPUT

```c
#include <stdio.h>

int main(void) {
    int diff;
    diff = 21;
    scanf("%ld",&diff); //Assume user enters 21474836484389
    printf("%d",diff);
    return 0;
}
```

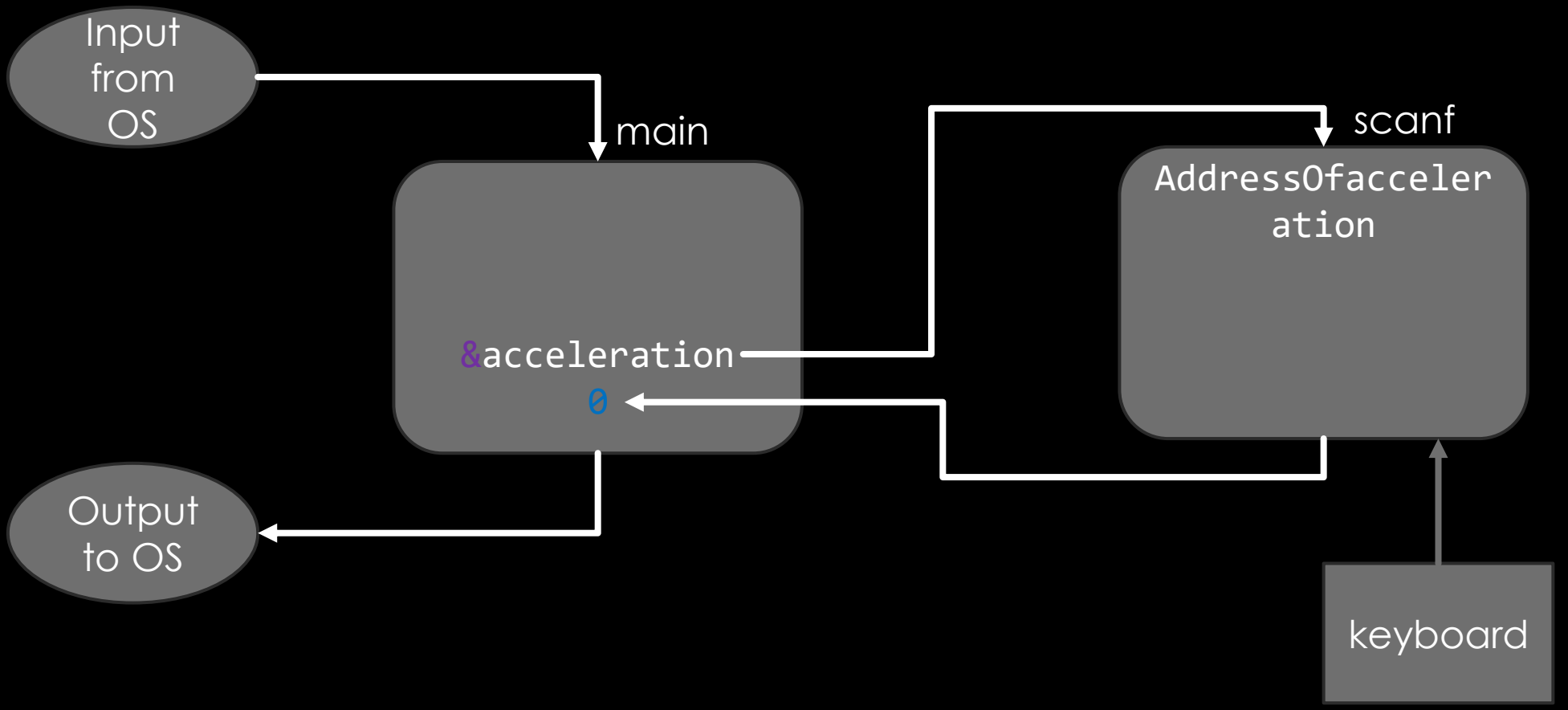
INPUTS ARE PASSED BY VALUE



INPUTS ARE PASSED BY VALUE



SIMULATING PASSING AN OBJECT (BY REFERENCE)



ALTERNATIVE COULD HAVE BEEN

```
principal=scanf("%f");  //But standard C function does not do this
```

MATH LIBRARY

Predefined Mathematical Constants, Trigonometric Functions and Inverses, Exponentiation and Logarithms, Hyperbolic Functions, Special Functions

FIND LARGEST NUMBER

```c
#include <stdio.h>
#include <math.h> //For standard math functions

int main(void) {
    double student1, student2, student3; // Names can have numbers but not at beginning
    printf("Enter marks of student1, student2, student3:\n");
    scanf("%lf,%lf,%lf",&student1, &student2, &student3);  //Formatted input
    printf("Max marks=%lf\n",                                    ); //Math function

}
```

FIND LARGEST NUMBER

```c
#include <stdio.h>
#include <math.h> //For standard math functions

int main(void) {
    double student1, student2, student3; // Names can have numbers but not at beginning
    printf("Enter marks of student1, student2, student3:\n");
    scanf("%lf,%lf,%lf",&student1, &student2, &student3);  //Formatted input
    printf("Max marks=%lf\n",fmax(fmax(student1,student2),student3)); //Math function
    return 0;
}
```